Course: Computer Vision

Course Code : ITA0506

Slot: D

Faculty: Jeni Gracia R J

Submitted By

Y Sravan Kumar (192472289)

CO5_ASSESSMENT 3

MOCK INTERVIEW QUESTIONS

1. What is OpenCV, and why is it commonly used in computer vision applications?

Answer:
OpenCV stands for **Open Source Computer Vision Library**. It is a Python/C++ library used for image and video processing, object detection, face recognition, tracking, and other computer vision tasks.

Key point: It is popular because it is **open-source, fast, and provides many ready-to-use computer vision functions**.

2. If you are given a folder containing hundreds of images, how would you design an OpenCV pipeline to read and process them efficiently?

Answer:
I would first get the image file paths from the folder, read each image using `cv2.imread()`, preprocess it, perform the required operation, save the result, and then move to the next image.

Pipeline:

```
Folder
  ↓
Get Image Paths
  ↓
Read Image
  ↓
Preprocess
  ↓
Process
  ↓
Save Result
```

```
  ↓
Next Image
```

For efficiency, I would avoid loading all images into memory at once.

3. What would you check if cv2.imread() returns None while reading an image?

Answer:

I would check whether the file path is correct, whether the file exists, whether the filename and extension are correct, and whether OpenCV has permission to access the file.

Example:

```
img = cv2.imread("image.jpg")

if img is None:
    print("Image could not be loaded")
```

4. Suppose a video is not opening using cv2.VideoCapture(). How would you systematically identify the problem?

Answer:

I would check:

1. Whether the video file path is correct.
2. Whether the file exists.
3. Whether the camera index is correct.
4. Whether the video format is supported.
5. Whether another application is using the camera.
6. Whether `cap.isOpened()` returns `True`.

Example:

```
cap = cv2.VideoCapture("video.mp4")
```

```
if not cap.isOpened():
    print("Unable to open video")
```

5. What is the difference between processing an image and processing a video stream in OpenCV?

Answer:
An **image is processed as a single frame**, while a video is a sequence of frames that must be processed continuously.

```
Image → One frame → Process → Output

Video → Frame 1 → Frame 2 → Frame 3 → ...
                      ↓
                Continuous processing
```

6. If your video-processing application is dropping frames, what could be the reasons, and how would you improve its performance?

Answer:
Frames may be dropped because the processing is too slow compared with the video frame rate.

I would improve performance by:

- Reducing frame resolution.
- Using a lightweight detection model.
- Processing only the required ROI.
- Skipping unnecessary frames.
- Optimizing image-processing operations.
- Using GPU acceleration if available.

7. How would you process only every 5th frame of a video? What are the advantages and disadvantages?

Answer:
I would maintain a frame counter and process the frame only when the counter is divisible by 5.

```
frame_count += 1

if frame_count % 5 == 0:
    process(frame)
```

Advantage: Reduces processing time.

Disadvantage: Important events between processed frames may be missed.

8. Why might an image appear darker after reading it in OpenCV, and how would you correct the problem?

Answer:
One common issue is incorrect handling of color channels, especially when displaying an image using libraries that expect RGB while OpenCV reads images in BGR format.

I would convert the image appropriately:

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

For actual brightness problems, I can also use brightness adjustment or histogram equalization.

9. If an OpenCV application has high processing latency, which stages of the pipeline would you investigate first?

Answer:
I would measure the time taken by each stage:

```
Image Acquisition
       ↓
Preprocessing
```

```
       ↓
Detection
       ↓
Feature Extraction
       ↓
Tracking
       ↓
Visualization
```

Usually, I would investigate **object detection and feature extraction first**, because complex models can consume significant processing time.

10. How would you design a modular OpenCV application in which preprocessing, detection, tracking, and visualization are independent modules?

Answer:
I would create separate functions or classes for each task.

```
main()
  │
  ├── capture_frame()
  ├── preprocess()
  ├── detect_objects()
  ├── track_objects()
  └── visualize()
```

This makes the program **easier to test, maintain, modify, and debug**.

IMAGE PROCESSING AND FEATURES

11. What is the difference between edge detection and corner detection? Give a practical application for each.

Answer:

Edge detection identifies boundaries where image intensity changes significantly.

Example: detecting the edges of a road or document.

Corner detection identifies points where two or more edges meet.

Example: detecting important points for image matching.

```
Edge → Boundary
Corner → Important intersection point
```

12. If a feature detector produces thousands of keypoints, how would you reduce the number of irrelevant features?

Answer:
I would filter keypoints based on their **response/strength**, remove weak features, restrict detection to a region of interest, and select the strongest or most useful keypoints.

13. How would you compare two images and determine whether they contain significant differences?

Answer:
I would first make the images comparable in size and format, then calculate their pixel difference.

A simple method is:

```
Image 1
   ↓
Absolute Difference
   ↑
Image 2
   ↓
Threshold
```

```
   ↓
Difference Regions
```

For more complex cases, I could compare features or use structural similarity.

14. Suppose feature matching works for two images taken from the same viewpoint but fails when one image is rotated. How would you improve the system?

Answer:
I would use **rotation-invariant feature descriptors**, such as SIFT or ORB, and use appropriate feature matching techniques.

This allows important features to be matched even when the image orientation changes.

15. How would you design a system to track important features from one video frame to the next?

Answer:
I would:

1. Detect important features in the first frame.
2. Extract their locations.
3. Track those points in subsequent frames.
4. Update their positions continuously.
5. Remove points that are lost.

A method such as **Lucas-Kanade optical flow** can be used for tracking feature points.

FACE RECOGNITION

16. What are the major challenges in face detection when multiple people are present in a scene? How would you address them?

Answer:
Major challenges include:

- Different face sizes.
- Different orientations.
- Occlusion.
- Poor lighting.
- Faces appearing close together.
- Background noise.

I would use a robust face detector, preprocessing, appropriate confidence thresholds, and multi-face detection.

17. How would you design a simple face-recognition system using OpenCV? Explain the complete pipeline from image acquisition to identification.

Answer:

```
Camera
  ↓
Frame Acquisition
  ↓
Preprocessing
  ↓
Face Detection
  ↓
Face Alignment
  ↓
Feature Extraction
  ↓
Compare with Registered Dataset
  ↓
```

```
Recognition
  ↓
Name + Confidence
```

The camera captures the image, faces are detected and preprocessed, facial features are extracted, and those features are compared with registered faces. If the similarity passes a threshold, the person's identity is displayed; otherwise, the person is marked as **Unknown**.

18. A face-recognition system incorrectly identifies an unknown person as a known person. What changes would you make to reduce false recognition?

Answer:
I would:

- Increase the recognition threshold.
- Use better-quality training images.
- Register multiple images per person.
- Improve face alignment.
- Reject low-quality faces.
- Require recognition across multiple frames.
- Mark the person as **Unknown** when confidence is insufficient.

Most important: Use a suitable **matching threshold**.

19. How would you improve face-recognition performance when images are captured under different lighting conditions?

Answer:
I would use preprocessing techniques such as:

- Histogram equalization.
- CLAHE.
- Brightness normalization.

- Noise reduction.

I would also train/register faces under different lighting conditions.

OCR AND DOCUMENT PROCESSING

20. What preprocessing techniques would you apply to a noisy scanned document before sending it to an OCR system?

Answer:

```
Scanned Document
       ↓
Grayscale
       ↓
Noise Removal
       ↓
Thresholding
       ↓
Morphological Operations
       ↓
OCR
```

I would use **grayscale conversion, median/Gaussian filtering, thresholding, and morphological operations** to improve text clarity.

21. If text in an image is rotated or captured at an angle, how would you prepare the image for OCR?

Answer:
I would use **deskewing and geometric/perspective correction**.

```
Tilted Document
      ↓
Detect orientation/corners
```

```
      ↓
Rotation/Perspective Correction
      ↓
Straight Document
      ↓
OCR
```

This makes the text easier for the OCR engine to recognize.

SURVEILLANCE AND MOTION DETECTION

22. How would you design a real-time system that detects moving objects in a surveillance video?

Answer:

```
Camera
  ↓
Video Frames
  ↓
Preprocessing
  ↓
Background Subtraction
  ↓
Morphological Filtering
  ↓
Contour Detection
  ↓
Object Detection
  ↓
Tracking
  ↓
Alert / Visualization
```

For a fixed camera, techniques such as **MOG2 or KNN background subtraction** can be used.

23. Suppose a surveillance system generates many false motion detections because of moving trees and changing shadows. How would you reduce these false alarms?

Answer:
I would:

- Apply Gaussian/median filtering.
- Use morphological operations.
- Ignore very small contours.
- Use adaptive background subtraction.
- Apply a region of interest.
- Analyze object size and movement.
- Use object detection to distinguish people from background movement.

For example:

```
Small movement → Ignore
Large persistent movement → Analyze
Person in restricted area → Alert
```

24. How would you design a people-counting system for a doorway? What problems could occur because of occlusion, people stopping, or changing direction?

Answer:

```
Camera
  ↓
Person Detection
  ↓
Person Tracking
```

```
  ↓
Virtual Line
  ↓
Line Crossing
  ↓
Count
```

Each person receives a unique tracking ID.

Problems

- Two people may overlap.
- A person may stop near the line.
- A person may change direction.
- A person may partially disappear.
- The same person may be counted twice.

Solutions

Use:

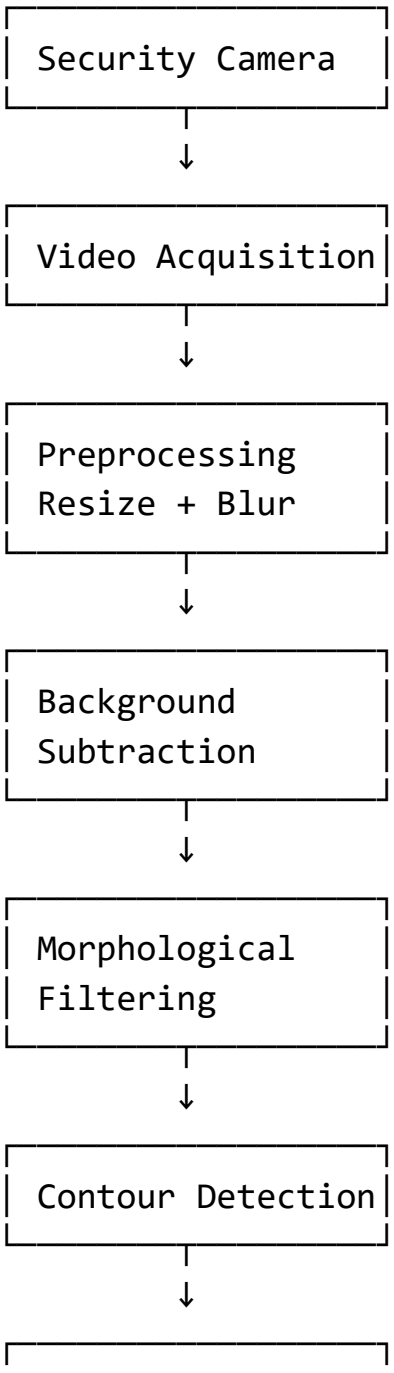
- Multi-object tracking.
- Unique tracking IDs.
- Center-point tracking.
- Direction analysis.
- A counted-ID list.
- Temporary tracking when a person disappears.

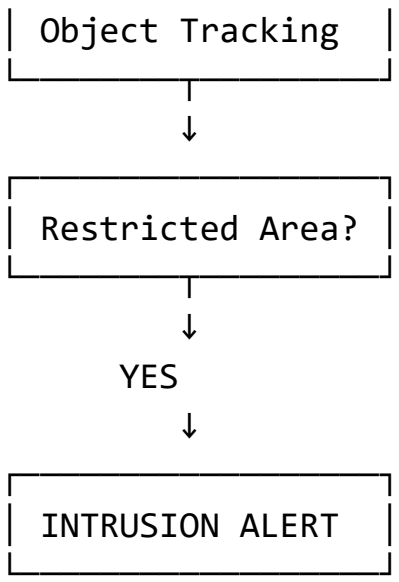
25. Imagine you are asked to develop a real-world Computer Vision application within one week using OpenCV. What application would you choose, and how would you design its complete pipeline?

Answer: Smart Home Intrusion Detection System

I would choose a **Smart Home Intrusion Detection System** because it can be developed using OpenCV within a short time and demonstrates several important computer vision concepts.

Complete Pipeline





Why I selected each technique

VideoCapture: Captures live camera frames.

Resize: Reduces computational requirements.

Gaussian Blur: Reduces camera noise.

Background Subtraction: Detects moving objects in a fixed-camera environment.

Morphological Operations: Removes small noise and fills gaps.

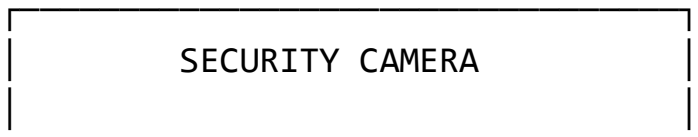
Contours: Identifies the boundaries of moving objects.

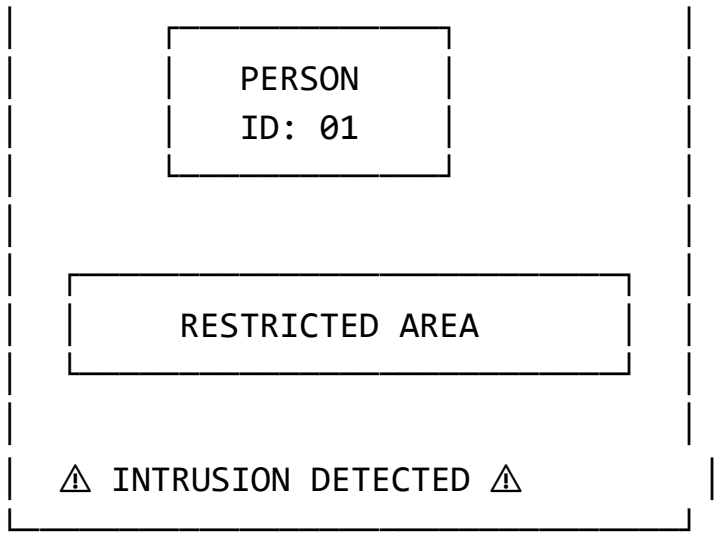
Tracking: Maintains the identity/location of an object across frames.

ROI: Restricts processing to important areas such as the entrance.

Alert: Notifies the user when a person enters the restricted region.

Expected Output




PERSON
ID: 01
RESTRICTED AREA
⚠ INTRUSION DETECTED ⚠